

CAN Communication & ECU Simulation

STM32 CAN communication layer + Vector CANoe virtual network simulation and validation

Project focus

Connect low-level CAN protocol understanding with embedded implementation, virtual ECU behavior, network simulation, and runtime validation using Vector CANoe.

STM32 / Embedded C

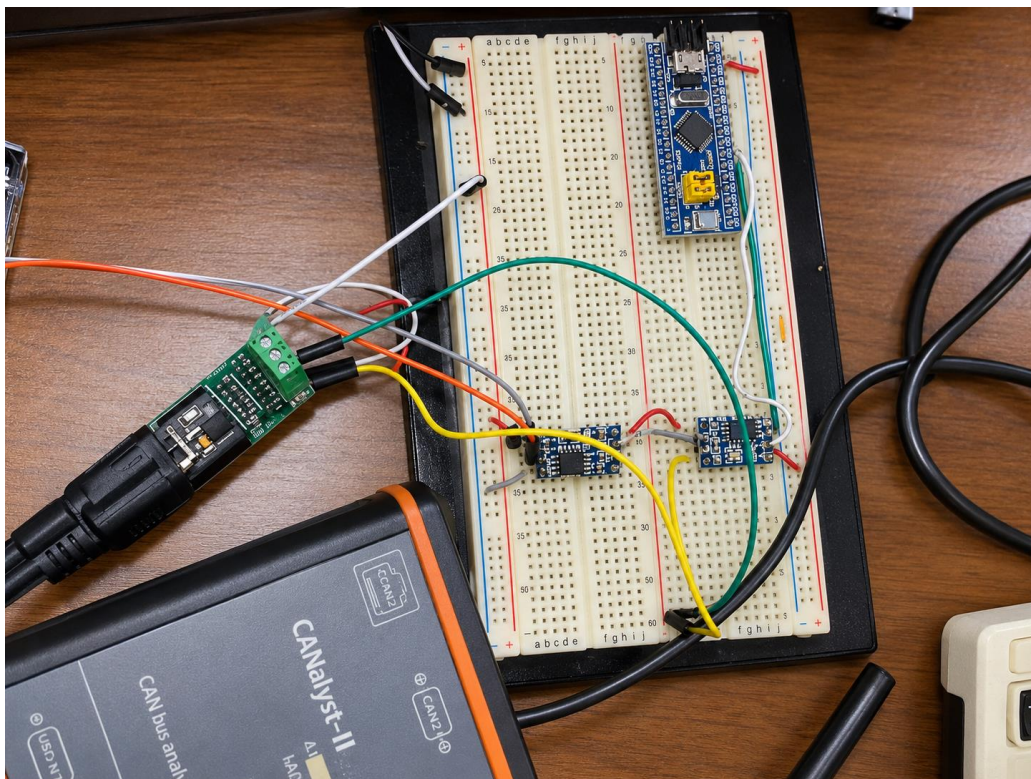
CAN Protocol & Messages

Vector CANoe Virtual Bus

CAPL Virtual ECUs

Trace + Graphics + Interactive Panel

Portfolio Case Study



1. Project Objective & Scope

The purpose of this project was to move beyond using CAN as a black-box peripheral. The work combined protocol study, STM32-side communication software, and a Vector CANoe environment that simulates multiple ECUs on a shared CAN bus.

Hands-on scope

- Studied CAN frame behavior, arbitration, ACK/error handling, filtering, FIFOs, and controller operation.
- Developed a CAN communication layer on STM32 using HAL APIs for frame transmission, reception, filtering, and application-level handling.
- Configured a Vector Virtual CAN Bus and CANoe application channel.
- Created two virtual ECUs using CAPL and exchanged CAN messages/signals between them.
- Built an interactive CANoe panel using System Variables and visualized runtime behavior through Trace, Write, and Graphics windows.

Physical integration scope

The physical CANoe integration workflow was additionally studied through a reference setup using a Vector CAN interface connected to real embedded hardware. This physical reference was used to understand channel configuration, CAN-H/CAN-L connectivity, live-bus observation, and how real ECU traffic appears inside CANoe.

Accuracy note

The virtual CANoe network and CAPL implementation are the hands-on implementation documented here. The physical bench images are presented as a studied/reference integration workflow.

2. CAN Protocol Knowledge Applied

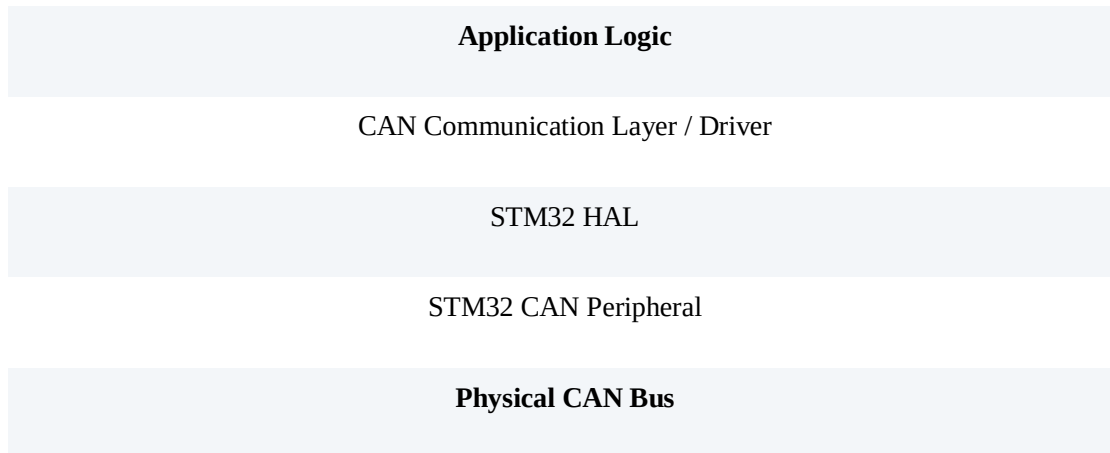
The CANoe work was built on top of a protocol-level understanding of Classical CAN. Key concepts covered during the development include:

- CAN data frame structure and standard/extended identifiers
- Dominant/recessive bit behavior and non-destructive arbitration
- ACK mechanism and transmitter/receiver behavior
- CRC, bit stuffing, error detection, and error frames
- Error Active, Error Passive, and Bus-Off states
- Bit timing / baud-rate concepts
- Acceptance filtering and receive FIFOs
- Data frames versus remote frames
- CAN-H / CAN-L differential physical layer concept
- Interrupt-driven transmit/receive flow inside the MCU

This layer of understanding is important because CANoe is not only a visualization tool; interpreting its Trace window, frame timing, signal values, and error behavior depends on understanding what the controller and bus are doing underneath.

3. STM32 CAN Communication Layer

On the embedded side, the CAN functionality was organized on top of STM32 HAL APIs rather than rewriting the complete peripheral driver from registers. This reduced boilerplate while still allowing the application to control CAN configuration and communication behavior.



Functions addressed

- CAN initialization and operating mode configuration
- Message header / identifier and DLC handling
- Frame transmission
- Frame reception
- Acceptance filter configuration
- Rx FIFO handling
- Application-level payload processing

Recommended portfolio wording

Developed an STM32 CAN communication layer using HAL APIs for CAN frame transmission, reception, filtering, and application-level communication.

4. Vector CANoe Virtual Network Setup

For software-only network simulation, CANoe was mapped to a Virtual CAN Bus. This allows multiple simulated ECUs to exchange CAN frames without requiring a physical Vector interface or external CAN transceiver during the virtual test.

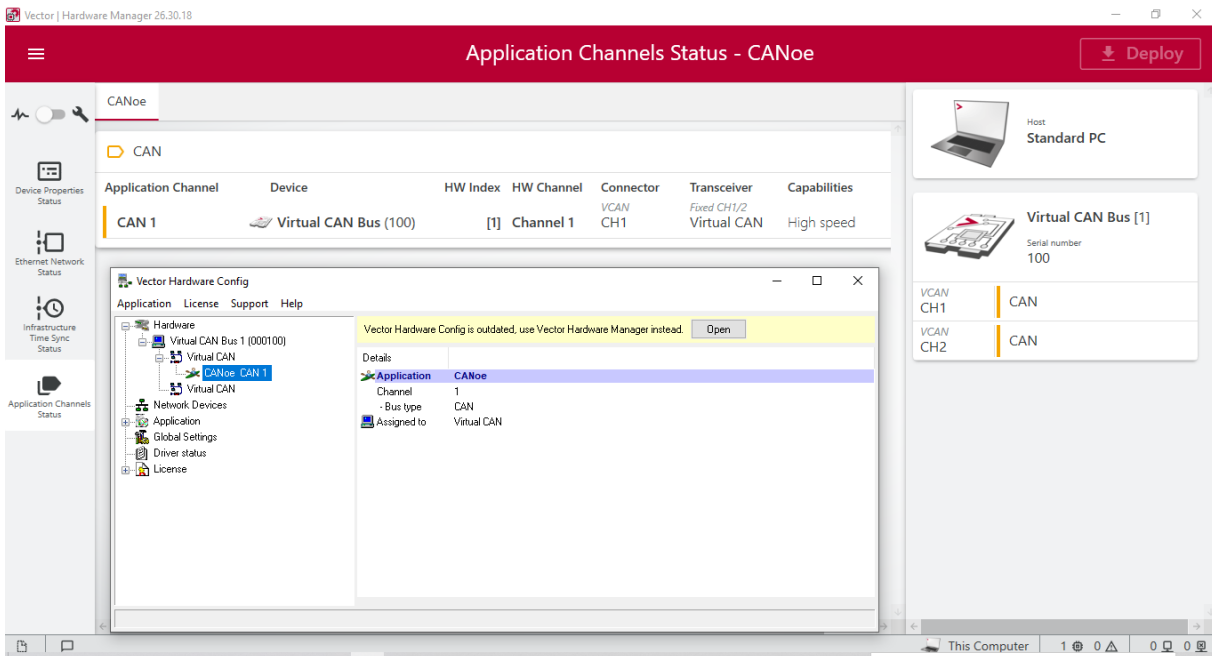


Figure 1. Vector Hardware Manager: CANoe application channel CAN 1 mapped to Virtual CAN Bus (100), Channel 1.

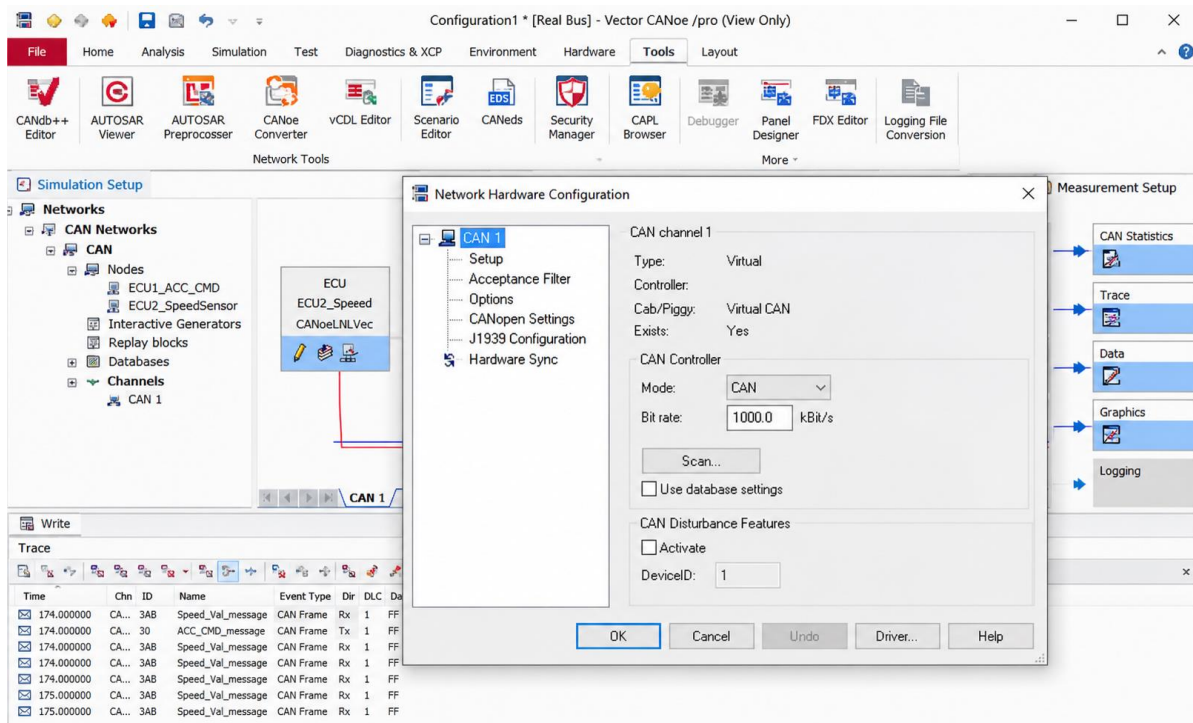


Figure 2. CANoe Network Hardware Configuration in Virtual mode. The screenshot shows a virtual CAN channel and configured bitrate.

The virtual network separates network behavior testing from physical hardware availability. It is therefore useful for early ECU logic development, message validation, and rest-of-bus style simulation.

5. Virtual ECU Topology

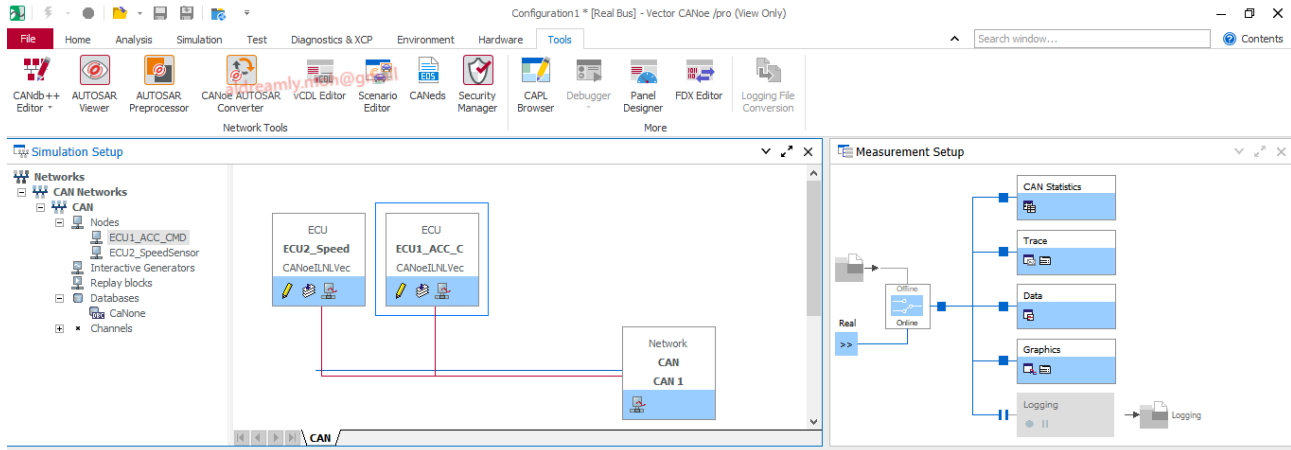


Figure 3. CANoe Simulation Setup with two nodes: ECU1_ACC_CMD and ECU2_SpeedSensor connected to CAN 1.

Two nodes were created to model a simplified automotive interaction. ECU1 acts as the ACC command source; ECU2 receives the command and models a changing speed value. Both are implemented in CAPL (Communication Access Programming Language), Vector's event-driven scripting language for CANoe/CANalyzer environments.

Interactive Panel / System Variable

ECU1_ACC_CMD (CAPL)

ACC_CMD CAN Message

ECU2_SpeedSensor (CAPL)

Speed_Val CAN Message

Panel + Trace + Graphics

6. CAPL ECU 1 - ACC Command Node

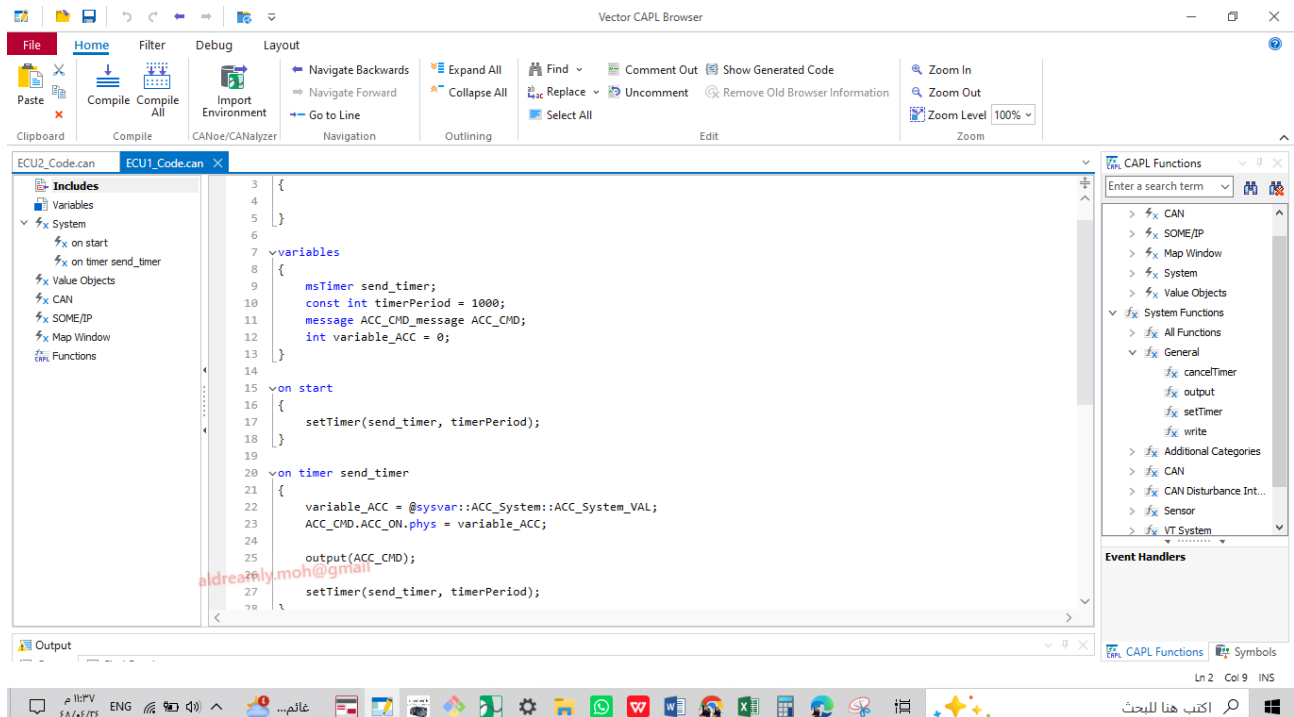


Figure 4. ECU1 CAPL code: periodic timer reads the ACC System Variable, assigns it to ACC_ON.phys, and transmits ACC_CMD.

ECU1 uses a periodic timer of 1000 ms. On every timer event, it reads the CANoe System Variable ACC_System::ACC_System_VAL, copies the value to the ACC_ON signal, transmits the ACC_CMD message, and rearms the timer.

Why a System Variable?

A CANoe System Variable is shared across the measurement environment. This makes it suitable for connecting the GUI Panel, CAPL nodes, and other CANoe components without hard-coding a local CAPL variable. The Panel therefore acts as a runtime input to ECU1.

Panel Switch

System Variable: ACC_System::ACC_System_VAL

CAPL Timer Event

ACC_ON Signal

ACC_CMD Message

7. CAPL ECU 2 - Speed Behavior Node

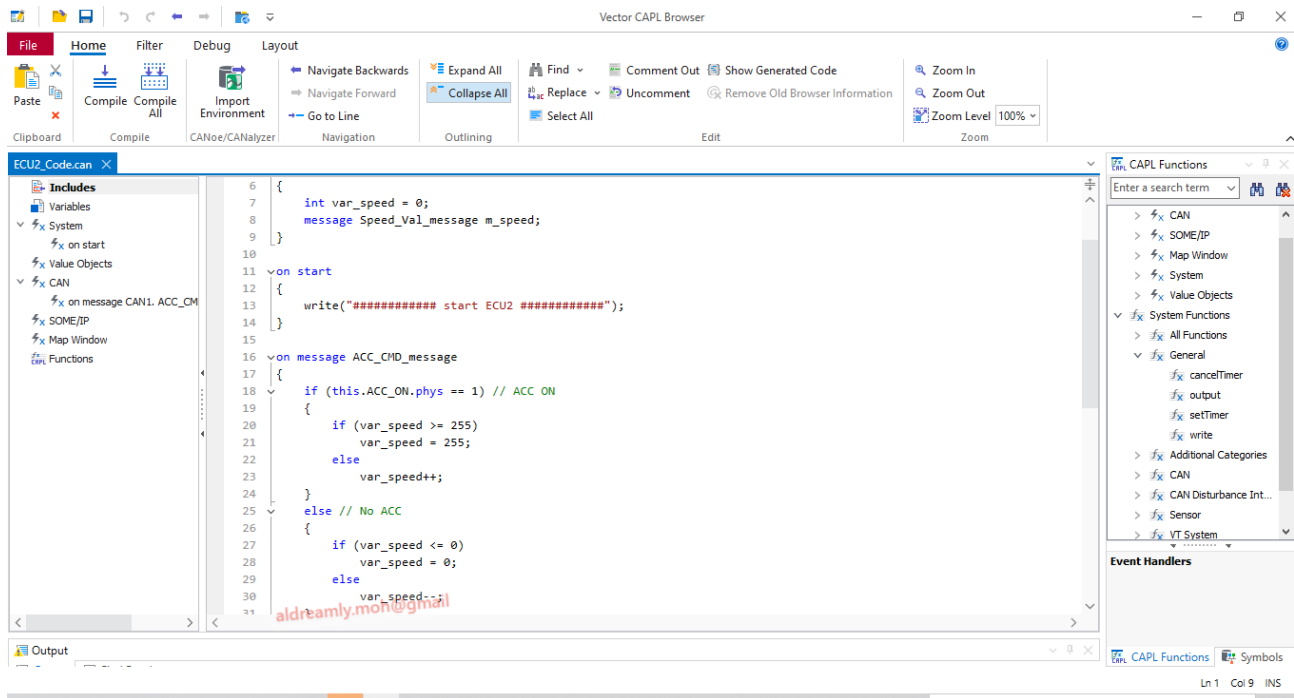


Figure 5. ECU2 CAPL handler for ACC_CMD_message. Speed is incremented/decremented and constrained to the allowed range.

ECU2 reacts to the received ACC command. When ACC_ON is active, the simulated speed increases. When the command is inactive, the speed decreases. Saturation logic prevents the simulated value from exceeding the configured limits (0 to 255 in this demonstration).

This is intentionally a simple acceleration/deceleration behavior model rather than a full closed-loop cruise-control algorithm. Its role is to demonstrate ECU interaction, CAN message handling, signal processing, and runtime validation.

8. CANoe Panel & System Variables

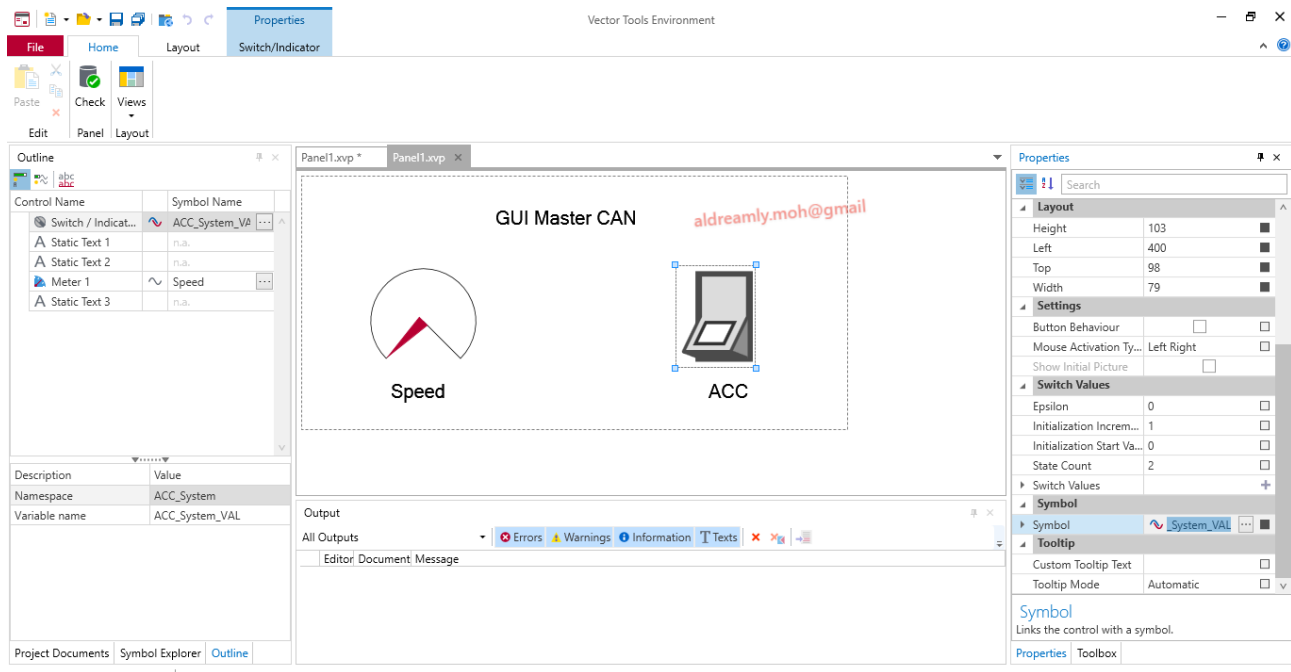


Figure 6. CANoe Panel Designer: ACC switch linked to the ACC System Variable and a Speed meter for live visualization.

The panel provides a human-machine interface for the simulation. The ACC control changes the shared System Variable, which is sampled by ECU1. The speed indicator provides immediate visual feedback from the simulated network state.

Skill demonstrated

CANoe Panel Designer + System Variables + signal binding + runtime interaction with CAPL nodes.

9. Runtime Validation: Trace, Graphics & Panel

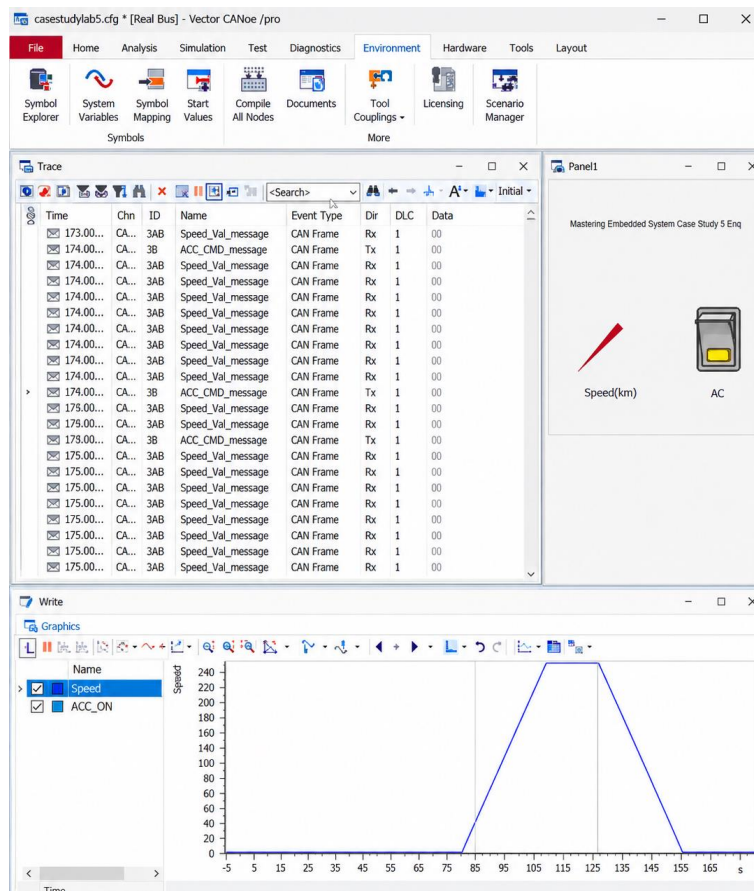


Figure 7. Runtime Trace and Graphics view showing ACC_CMD_message and Speed_Val_message traffic with acceleration/deceleration behavior.

Figure 8. Runtime panel and Write output during the simulation; the speed reaches the configured maximum value.

The Trace window confirms that both CAN messages are present on the virtual bus. The Graphics view provides a time-based view of the speed signal, while the panel and Write output give immediate feedback about the current simulation state.

Validation performed

- Message presence on CAN 1
- Signal/state changes following ACC input
- Periodic command transmission
- Speed increase/decrease behavior
- Upper and lower saturation behavior
- Real-time visualization through Trace, Graphics, Write, and Panel windows

10. Physical CANoe Integration - Reference Workflow

After validating the all-virtual network, the next architecture is a mixed real/virtual network: a physical ECU under test connected to CANoe through a Vector interface, while the remaining ECUs can stay virtual. This is commonly used as a rest-of-bus testing approach.

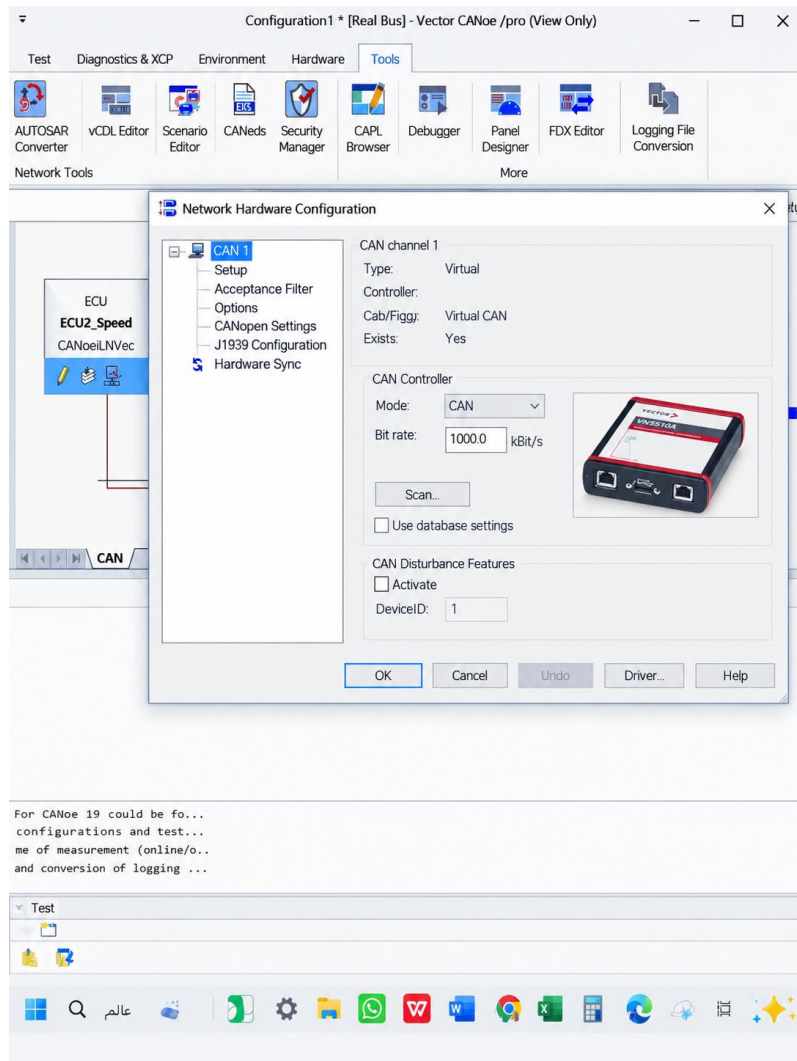


Figure 9. Reference physical CANoe configuration using a Vector VN5610A CAN/CAN FD interface and a real CAN channel.

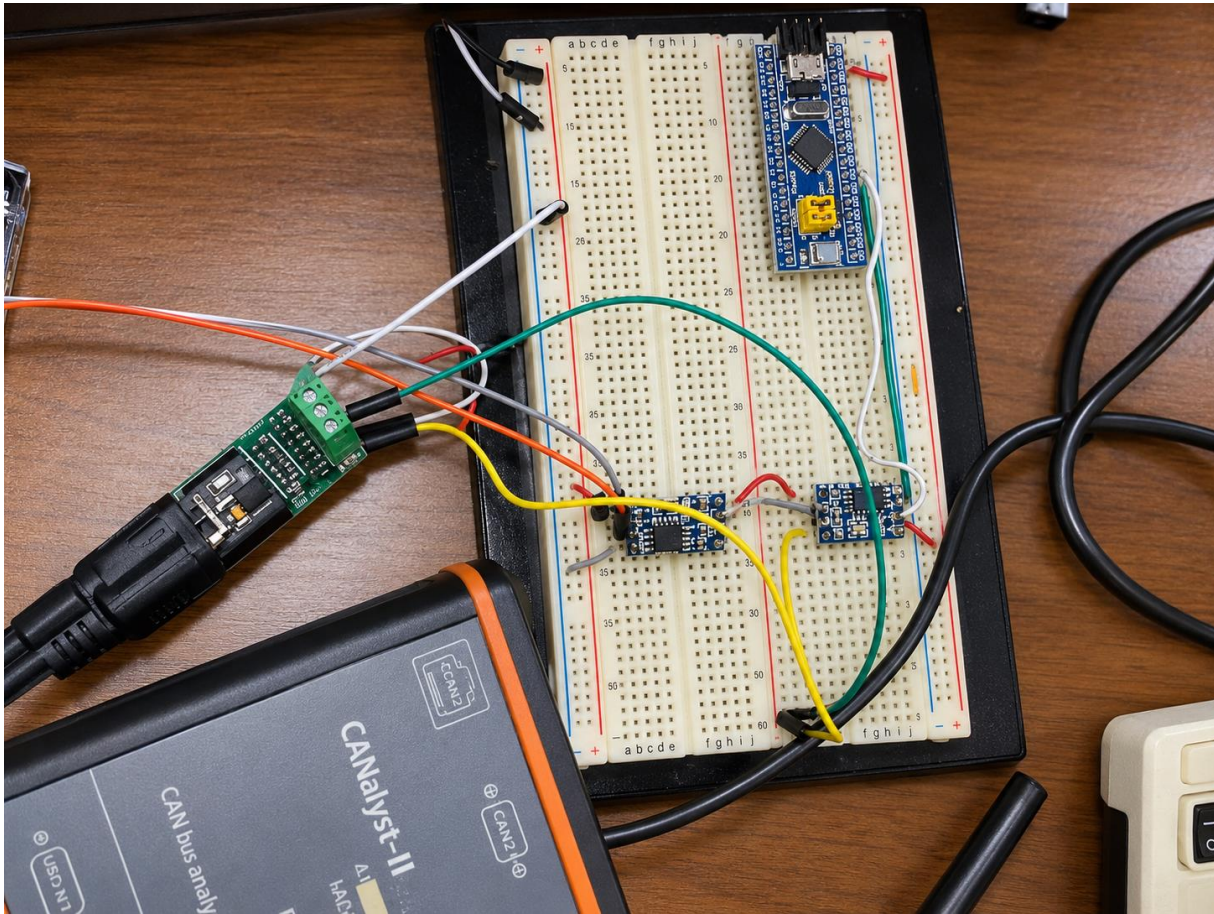


Figure 10. Reference physical bench showing embedded hardware and CAN interface wiring used to illustrate real-bus integration.

In the physical case, the Vector interface provides the bridge between the PC/CANoe environment and the electrical CAN bus. CAN-H and CAN-L are connected to the target ECU/transceiver network, and CANoe can then observe or stimulate real traffic depending on the configured test setup.

10.1 Mixed Real/Virtual Architecture

PC + Vector CANoe

Vector CAN Interface (e.g., VN-series)

CAN-H / CAN-L

Physical STM32 ECU Under Test

Virtual ECUs / Rest-of-Bus Nodes in CANoe

The important concept is that the complete vehicle network does not need to exist physically. One real ECU can be connected to the bus while CANoe simulates the remaining nodes, sends expected traffic, and monitors the response of the ECU under test.

How this should be described publicly

Studied and verified the physical CANoe integration workflow through a reference bench setup, including Vector interface channel mapping, real CAN bus connectivity, and live message observation. The hands-on implementation in this case study is the virtual CANoe/CAPL network.

11. Engineering Takeaways & Skills Demonstrated

The main value of the exercise is the connection between embedded implementation and network-level validation. Instead of treating CAN communication as an isolated driver task, the project follows the message from the MCU/software layer into a multi-ECU network simulation and finally into measurement/visualization tools.

Area	Evidence
CAN Protocol	Frames, arbitration, ACK/error concepts, filtering, FIFOs, timing
Embedded C / STM32	CAN communication layer using STM32 HAL
Vector CANoe	Virtual bus setup, channels, measurement environment
CAPL	Event-driven virtual ECU behavior and message handling
CANoe Panel	System Variable binding and interactive controls
Network Validation	Trace, Write, Graphics, signal behavior and timing
System Thinking	Virtual ECUs, physical ECU concept, rest-of-bus workflow

Technology keywords

STM32 • Embedded C • CAN Protocol • STM32 HAL • Vector CANoe • CAPL • CAN Messages & Signals • System Variables • Virtual ECUs • Rest-of-Bus Simulation • CAN Trace Analysis • Network Testing